

LangSmith: Observability & Evaluation for LLM Applications

Why It's Needed, Core Concepts, and Integration with LangChain & LangGraph

1 Why LangSmith? Three Motivating Scenarios

LLM-based systems are hard to debug in production because LLM behavior is **non-deterministic** (the same or similar input can produce different outputs each time) and because these systems are often complex, multi-stage pipelines that behave like **black boxes** once deployed. The following three scenarios illustrate the kinds of problems that show up.

1.1 Scenario 1: Increased Latency in a Complex LLM Workflow

A startup builds a tool that helps students apply for jobs: it studies a job description (JD), fetches the student's resume/portfolio from Google Drive, matches relevant skills, and generates (and proofreads) a tailored cover letter. Normally this takes about 2 minutes end-to-end.

One day, users start complaining that the tool has become very slow (7–10 minutes instead of 2). The workflow has multiple LLM-dependent stages (reading the JD, matching skills, generating the letter, proofreading), but the only visibility available is the initial input, the final output, and the total time taken. There's no way to see **which specific stage** is responsible for the extra delay (e.g., a bug that causes the whole Google Drive to be scanned instead of a specific folder). Without a way to break the execution down stage by stage, this kind of debugging is extremely difficult.

1.2 Scenario 2: Unexpected Cost Spike in an Agent

A research assistant agent takes a topic, fetches relevant academic papers (e.g., from Google Scholar or arXiv), extracts key points from each, and summarizes them into a report – at a cost of roughly 50 paise per report.

After a prompt change intended to improve report quality (“keep refining until the report is really good”), the agent starts looping back through the entire pipeline for some topics when it isn't satisfied with its own output. This causes some reports to cost up to Rs. 2 instead of 50 paise – inconsistently, and without any clear error trace. Since agents reason and act autonomously in a loop, and there's no visibility into *why* the agent chose to repeat its process for some queries but not others, this is very hard to debug without an inside view of the system.

1.3 Scenario 3: Hallucination in a RAG-Based Chatbot

A company builds a RAG-based internal chatbot to answer employee questions (leave policy, notice period, health insurance, etc.) from company documents, reducing HR's repetitive workload.

Over time, the chatbot starts **hallucinating** – giving fabricated, factually wrong answers (e.g., incorrectly telling an employee there’s no leave policy limit) – causing serious misinformation risk. A RAG system can hallucinate for essentially two reasons:

- **Retriever failure** – the retriever fetches documents that aren’t actually relevant to the question (e.g., an overly small k/n value for retrieved documents).
- **Generator failure** – the LLM itself answers poorly, e.g., due to a weak prompt that doesn’t strongly enforce “only answer from the given context,” or a lower-quality/misbehaving model.

Without visibility into what the retriever actually fetched and what the final prompt (question + context) looked like, it’s impossible to tell whether the retriever or the generator is at fault.

1.4 The Common Thread

All three examples are LLM-based systems, and all three needed a way to open up the black box and understand, component by component, what’s actually happening inside the system during execution. This is exactly the gap that a tool like LangSmith fills.

2 Observability

Definition: Observability is the ability to understand a system’s internal state by examining its external outputs, such as logs, metrics, and traces. It allows you to diagnose issues, understand performance, and improve reliability by analyzing data generated by the system – essentially, being able to answer *why* something is happening within a system, even if you didn’t anticipate the problem.

Because LLM-based systems are non-deterministic (same input, potentially different output) and don’t leave behind clean, traditional error traces, observability – being able to open up the system and see what each component is doing – becomes essential once such systems are deployed in production.

2.1 What is LangSmith?

Definition: LangSmith is a unified observability and evaluation platform where teams can debug, test, and monitor AI app performance.

In short, LangSmith lets you bring the concept of observability into your LLM-based applications: you run/execute your LLM application, and LangSmith traces that entire execution component by component, recording exactly what each component received as input, what it produced as output, and how long it took – all at a very granular level.

2.2 What Does LangSmith Trace?

For every execution it logs, LangSmith captures:

- The overall **input and output** of the run.
- All **intermediate steps/states** (e.g., in a RAG system: what question the retriever received, what context it generated, what exact prompt was sent to the LLM, what the LLM produced, what the output parser did with it).

- **Latency** – both at the whole-application level and at the individual component level.
- **Token usage** and the associated **cost** (for both input and output tokens), based on the model used.
- Any **errors** that occurred in any component.
- **Tags** – both automatically generated ones (e.g., model name for GPT-4o) and custom ones you define.
- **Metadata** – both system-generated (e.g., library versions) and custom metadata you attach.
- Optional **user feedback** attached to a trace.

3 Setup

1. Clone the example code repository and open it in an editor.
2. Create and activate a virtual environment, then install dependencies from `requirements.txt`:

```
python -m venv myenv
myenv\Scripts\activate
pip install -r requirements.txt
```

3. Sign up / log in at the LangSmith website, go to **Settings**, and generate a **Personal Access Token** API key.
4. Create a `.env` file in the project with:

```
OPENAI_API_KEY=your_openai_key
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=<langsmith_endpoint_url>
LANGCHAIN_API_KEY=your_langsmith_key
LANGCHAIN_PROJECT=your_project_name
```

5. The value given for `LANGCHAIN_PROJECT` determines the project name under which all traces for this code will be grouped in the LangSmith UI.

4 Core Concepts: Project, Trace, Run

Consider a simple LLM app: take a user’s question → format it into a prompt → send it to an LLM → parse the LLM’s output → show it to the user. This is implemented as a chain of three components: a **prompt template**, a **model**, and an **output parser**.

- **Project:** The entire LLM application itself. Whatever name is given as `LANGCHAIN_PROJECT` becomes the project under which all runs are grouped.
- **Trace:** A **single end-to-end execution** of the project. Every time the app runs (e.g., once per user query), that whole execution is one trace.
- **Run:** The execution of an **individual component** within a trace. In the example above, one trace consists of three runs – one for the prompt template, one for the model, one for the parser.

5 Code Demo 1: A Simple LLM Call

A minimal LangChain chain (prompt → model → parser) is built and run:

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template('{question}')
model = ChatOpenAI()
parser = StrOutputParser()

chain = prompt | model | parser
result = chain.invoke({'question': 'What is the capital of Peru?'})
print(result)
```

Key point: No LangSmith-specific code is required in the chain itself. Because the environment variables (LANGCHAIN_TRACING_V2, endpoint, API key, project name) are already set, LangSmith **automatically** traces the entire execution the moment the code runs.

In the LangSmith UI (**Tracing Projects**), the configured project shows one trace per execution, with:

- The overall input/output, any errors, and total latency.
- Token usage and cost.
- A breakdown into individual **runs** (prompt template, chat model, output parser), each independently inspectable for its own input, output, and time taken.

6 Code Demo 2: A Sequential Chain (Report → Summary)

A two-step chain generates a detailed report on a topic, then summarizes that report into five points – two sequential LLM calls.

```
prompt1 = PromptTemplate.from_template(
    'Generate a detailed report on the topic {topic}'
)
prompt2 = PromptTemplate.from_template(
    'Generate a five point summary from the following text \n {text}'
)

model = ChatOpenAI()
parser = StrOutputParser()

chain = prompt1 | model | parser | prompt2 | model | parser
```

6.1 Setting a Project Name from Code

Rather than editing the `.env` file, the project name can be set (and will override the `.env` value) directly in code:

```
import os
os.environ['LANGCHAIN_PROJECT'] = 'Sequential LLM App'
```

6.2 Using Multiple Models

```
model1 = ChatOpenAI(model='gpt-4o-mini', temperature=0.7)
model2 = ChatOpenAI(model='gpt-4o', temperature=0.5)
# model1 used for report generation, model2 for summarization
```

6.3 Attaching Custom Tags, Metadata, and a Run Name

When invoking the chain, a config dictionary can carry tags, metadata, and a custom run name:

```
config = {
    'tags': ['llm app', 'report generation', 'summarization'],
    'metadata': {
        'model1': 'gpt-4o-mini',
        'model1_temperature': 0.7,
        'parser': 'StrOutputParser'
    },
    'run_name': 'sequential chain'
}

result = chain.invoke({'topic': 'Unemployment in India'}, config=config)
```

In the LangSmith UI, this trace now shows under the new project (**Sequential LLM App**), with:

- A custom **run name** (`sequential chain`) instead of the auto-generated `RunnableSequence`.
- The custom **tags** and **metadata** attached at the trace level.
- Per-run metadata as well (e.g., each `ChatOpenAI` run shows exactly which model it used).
- Step labels within the sequence (e.g., “Step 1”, “Step 2”, ... “Step 5”) showing the position of each run in the overall chain.

7 Tracing a RAG Application

7.1 Why RAG Tracing Matters

A RAG (Retrieval-Augmented Generation) app answers a query using both the query itself and retrieved context. When answer quality is poor, the failure could come from either:

- The **retriever** (wrong/irrelevant chunks fetched), or
- The **generator** (LLM hallucinating despite good context).

Without visibility into intermediate steps, it’s impossible to tell which one is at fault. LangSmith solves this by tracing every intermediate step: the question, what the retriever fetched, the final assembled prompt (question + context), and the final LLM response.

7.2 Example RAG App

A simple RAG app answers questions about a PDF book (*An Introduction to Statistical Learning*) using:

1. **Loading** the PDF (PyPDFLoader).
2. **Chunking** it (chunk size 1000, chunk overlap 150).
3. **Embedding** the chunks and building a **retriever**.
4. A chain: a **parallel** step that passes the original question through unchanged while simultaneously sending it to the retriever (whose output is then merged/formatted into a single context string) → a **prompt** (with system instruction “answer only from the provided context; if not found, say you don’t know”) that takes both question and context → the **LLM** → an **output parser**.

7.3 Problem 1: Only Part of the App Gets Traced

By default, LangSmith automatically traces LangChain **Runnables** – i.e., wherever an `invoke()`-style call happens. The PDF loading, chunking, and embedding steps in the setup code don’t use runnables, so by default they are **not** traced at all – only the downstream chain (retrieval + prompt + LLM + parser) shows up. This means important information, like how long loading/chunking/embedding took, is invisible.

7.4 Fix: The @traceable Decorator

LangSmith’s `traceable` decorator lets you trace **any** regular Python function, even if it contains no LangChain runnables. The setup logic is refactored into separate functions, each decorated:

```
from langsmith import traceable

@traceable(name='Load PDF')
def load_pdf(path):
    ...
    return documents

@traceable(name='Split Documents')
def split_documents(documents, chunk_size, chunk_overlap):
    ...
    return chunks

@traceable(name='Build Vector Store')
def build_vector_store(chunks):
    ...
    return retriever

@traceable(name='Setup Pipeline')
def setup_pipeline(pdf_path):
    documents = load_pdf(pdf_path)
    chunks = split_documents(documents, chunk_size=1000, chunk_overlap=150)
    retriever = build_vector_store(chunks)
    return retriever
```

A custom run name for the final chain invocation can also be set:

```
config = {'run_name': 'PDF RAG Query'}
```

```
result = chain.invoke(question, config=config)
```

Now the LangSmith UI shows two traces for one overall app run: **Setup Pipeline** (containing Load PDF, Split Documents, Build Vector Store as separate, independently inspectable runs – e.g., Load PDF shows the file path in, and 441 page-documents out) and **PDF RAG Query** (the retrieval + prompt + LLM + parser chain). This is better, but ideally both should appear **nested under one single top-level trace** rather than as two separate, disconnected traces – since it’s really one application.

7.5 Custom Tags/Metadata on Traceable Functions

Just like with chain configs, individual `@traceable` functions can carry their own tags and metadata, which also makes them **searchable** in the UI later:

```
@traceable(name='Load PDF', tags=['pdf', 'loader'],
           metadata={'loader': 'PyPDFLoader'})
def load_pdf(path):
    ...

@traceable(name='Build Vector Store', tags=['embedding', 'vector_store'],
           metadata={'embedding_model': 'text-embedding-3-small'})
def build_vector_store(chunks):
    ...
```

7.6 Problem 2: Redundant Work on Every Run (Latency)

As built so far, **every** run of the app reloads the PDF, re-chunks it, and re-generates embeddings from scratch – even though nothing changed. This makes every query slow (e.g., 200 seconds), which isn’t logical.

7.7 Fix: Caching/Reusing an Index

The app is modified to use a vector store (e.g., FAISS) with an on-disk **index**:

- On the **first** run, no index exists yet, so it builds one (load → chunk → embed) and saves it to a local index directory.
- On **subsequent** runs, it checks whether a valid index already exists; if so, it **loads** the existing index directly instead of rebuilding it from scratch, making queries dramatically faster (e.g., dropping from 200 seconds down to a couple of seconds).

The index is **rebuilt** (rather than reused) only when something meaningful changes, such as:

- It’s the very first run (no index present yet).
- The PDF’s content, size, or last-modified time changes (i.e., a different or updated source file).
- Chunking parameters change (chunk size or chunk overlap).
- The embedding model used changes.

In the LangSmith trace, this shows up as either a **Build Index** run (first time / when inputs changed) or a **Load Index** run (subsequent, fast runs) at the top of the setup pipeline. This caching pattern – build once, reuse thereafter – is standard practice for production RAG systems.

8 Tracing an Agent Application

Agents are autonomous software applications: given a goal, they reason, take an action, observe the result, decide whether the goal is achieved, and loop until it is (the classic **Thought** → **Action** → **Observation** pattern, as in a ReAct-style agent).

When such an agent (e.g., one with a web search tool and a weather-lookup tool) is run and traced, LangSmith shows, step by step:

- The agent’s **scratchpad** being initialized (empty at first).
- The **prompt** constructed for the LLM (including tool descriptions, the Thought/Action/Observation format instructions, and the user’s question).
- Each **Thought** → **Action** → **Observation** cycle: what the LLM decided to do (e.g., “search DuckDuckGo for X”), the tool call and its result, and how that result gets appended back into the scratchpad before the next LLM call.
- This repeats – potentially invoking multiple different tools across several cycles for more complex queries (e.g., first searching for a fact, then using a second tool based on what was found) – until the agent produces a **final answer**.

Because every intermediate reasoning step, tool call, and scratchpad update is traced, LangSmith makes an otherwise fully autonomous, opaque agent loop transparent and debuggable – including total token usage, cost, and latency per step.

9 LangGraph + LangSmith Integration

LangGraph represents any LLM workflow as a **graph** of nodes (tasks) connected by edges (execution order), with a shared **state**. As graphs get more complex (branches, parallel paths, subgraphs), debugging them without visibility becomes very difficult – and since LangSmith and LangGraph are built by the same team, the integration is very tight.

9.1 Two Key Concepts

1. Executing a LangGraph graph, top to bottom, produces exactly **one trace** in LangSmith.
2. **Each node** in the graph becomes **one run** inside that trace. If a graph has five nodes, the trace will contain (at least) five corresponding runs.

Branching, conditionals, parallel execution, and subgraphs within the LangGraph graph are all captured and reflected in the trace structure.

9.2 Example: An Essay-Checking Graph

A LangGraph workflow takes an essay as input and evaluates it on three parameters – **language**, **analysis**, and **clarity of thought** – via three parallel nodes, each producing a feedback string and

a score. A fourth, **final evaluation** node combines the three feedbacks into an overall summary and averages the three scores.

- As usual, LangGraph automatically traces each **node**.
- Additionally, the function *inside* each node can also be wrapped with `@traceable`, so both the node itself and its underlying function show up in the trace – this is optional but gives an extra level of granularity.
- A **config** dictionary (with tags, metadata, and a custom run name) can be passed to `.invoke()` just like with LangChain chains.

In the LangSmith UI, the resulting trace shows the three evaluation nodes executing **in parallel**, each feeding into the final evaluation node. Drilling into any node shows its function’s input (the full graph **state**) and output (a partial state update). Nodes using a **structured-output LLM** (one bound to a Pydantic/schema-based output) show an internal **RunnableSequence** with a **RunnableLambda** step that extracts and shapes the LLM’s structured response according to the schema; nodes using a plain (unstructured) LLM call show a much simpler run. Per-node latency (e.g., time taken by the “evaluate analysis” node) and per-node cost are both visible.

Summary: LangGraph execution → one trace. Each node → one run. Nodes/functions can be named, tagged, and annotated with metadata just like LangChain components, making even complex agentic LangGraph workflows straightforward to debug and understand.

10 Beyond Observability: Other LangSmith Features

LangSmith is not limited to observability (tracing a single execution). It also supports several other capabilities, broadly falling under what’s known as **LLM Ops**.

10.1 Monitoring & Alerting

Definition: Monitoring in LangSmith looks across many traces at once to track the overall health of an LLM system. It aggregates key metrics like latency, token usage, cost, error rates, and success rates. Alerts can be configured to notify you when these metrics drift outside acceptable ranges.

- **Observability** = studying a **single** trace in depth.
- **Monitoring** = studying **many traces together** over time (e.g., average latency/cost/token-usage per day, trends across all executions) via dashboards/graphs (traces per day, latency, error rate, LLM call counts and latency, cost and tokens per trace, tool usage, etc.).
- **Alerting** = setting a threshold on a tracked metric (e.g., “latency > 5 seconds”) so that when it’s crossed, a notification/alert is automatically raised – catching problems as early emerging **patterns across runs** (rather than a one-off blip in a single trace) before they impact many users at scale.

10.2 Evaluation

Definition: Evaluation in LangSmith helps you systematically measure the quality of your LLM outputs. You can run tests against gold-standard datasets or apply custom evaluation metrics such as faithfulness, relevance, and completeness. LangSmith supports multiple approaches: automated

scoring with LLM-as-a-judge, semantic similarity checks, and custom Python evaluators, both online and offline.

Because LLMs are non-deterministic, even a small change (a prompt tweak, a different model, a different retriever) can significantly change outputs for the same input. Evaluation gives an objective, repeatable way to benchmark a system’s outputs against a standardized dataset before deciding to deploy a new version, helping confirm that a new version is actually better and preventing silent regressions. LangSmith provides both prebuilt evaluators (e.g., for hallucination, conciseness, code correctness) and support for fully custom evaluators.

10.3 Prompt Experimentation

Since prompt quality heavily affects LLM performance (the basis of prompt engineering as a discipline), it’s important to be able to systematically compare prompt variants rather than judging informally. LangSmith’s **Playground** lets you set up a prompt, an evaluation schema/criteria, and even test the same prompt across different models – effectively enabling **A/B testing of prompts** against a dataset, with results tracked over time to show which prompt variations worked best and under what conditions.

10.4 Prompt Versioning / Hub

Prompts can be stored, versioned, and collaborated on within LangSmith (with a public prompt hub also available to browse), giving teams a central place to host and manage prompts.

10.5 Dataset Creation & Annotation

Evaluation requires a standardized dataset. LangSmith provides tools to build such datasets for evaluation and fine-tuning – either by importing existing rows or starting from an empty dataset and adding entries (e.g., directly from a trace via “Add to dataset”). Manual **annotation**/labeling (marking whether a given response was correct or not) is supported, and dataset versions are stored for reuse across projects.

10.6 User Feedback Integration

Similar to a thumbs-up/thumbs-down mechanism (as seen in tools like ChatGPT), LangSmith lets you capture user ratings or structured feedback in production. Feedback is logged alongside the corresponding trace (tied to the exact prompt, model, and state used) and supports bulk analysis of what users liked or disliked, visible in the Monitoring tab as a feedback-score trend across traces.

10.7 Collaboration

LangSmith is designed for teams to work together effectively – replacing older, informal workflows (like emailing screenshots of a problematic execution). Any trace can be shared via a direct link so a teammate can view and study the exact same trace on their own machine. Prompt experiments can be versioned with invited collaborators, and custom dashboards can be built and shared as well.

11 Summary Table

Concept	Definition
Observability	Understanding a system’s internal state via its external outputs (logs, metrics, traces); answers “why” something happened
LangSmith	A unified observability and evaluation platform for debugging, testing, and monitoring AI application performance
Project	The overall LLM application being traced in LangSmith
Trace	One complete end-to-end execution of the project
Run	The execution of one individual component within a trace
@traceable	A LangSmith decorator that lets any Python function be traced, even without LangChain runnables inside it
Tags / Metadata	Custom labels and key-value data attached to a trace, run, or traceable function to organize and search traces
Retriever Error	A RAG failure mode where the retriever fetches irrelevant or insufficient context for the question
Generator Error	A RAG failure mode where the LLM produces a poor/hallucinated answer despite having good context
Monitoring	Aggregating metrics (latency, cost, tokens, error rate) across many traces over time to track overall system health
Alerting	Automated notification when a monitored metric crosses a defined threshold
Evaluation	Systematically measuring LLM output quality against datasets and metrics (e.g., faithfulness, relevance)
LLM-as-a-Judge	An evaluation approach where one LLM is used to score/evaluate the output of another LLM-based system
Prompt Experimentation	Systematically comparing (A/B testing) different prompt versions against a dataset and evaluation criteria
Dataset Annotation	Manually labeling/rating dataset entries (e.g., correct/incorrect) for use in evaluation
LLM Ops	The broader discipline covering observability, monitoring, evaluation, and related practices for running LLM systems reliably in production